

Reporte del Proyecto SIA

Gestión de Asistencia

Asignatura	INF2236 - Programación Avanzada
Profesor	Claudio Cubillos
Periodo	1er Semestre 2026
Integrantes	Nicolás Jaramillo, Lucas Ahumada, Alessandro Reginato
Fecha de entrega	11 de Septiembre de 2026
Versión del proyecto	Primera Entrega

Resumen del proyecto

Problema que aborda el sistema: Gestión de Asistencia de un año escolar para un colegio

Usuarios del sistema: Funcionarios Escolares

Funcionalidades principales: Organizar y digitalizar asistencia para un colegio

Tecnologías utilizadas: JAVA 11, Github, Netbeans 21, CSV

Forma de ejecución: Consola y Ventana

Tabla de Contenidos

Sección	Página
Tabla de contenidos	1
SIA-1. Análisis de datos y funcionalidades	2
SIA-2. Diseño conceptual de clases	3
SIA-3. Buenas prácticas	4
SIA-4. Colecciones anidadas	5
SIA-5. Sobrecarga de métodos	6
SIA-6. Sobreescritura de métodos	7
SIA-7. Menú de inserción y listado	8
SIA-8. Menú de edición, eliminación y búsqueda	9
SIA-9. Funcionalidad de utilidad para el negocio	10
SIA-10. Consola y ventanas	11
SIA-11. Persistencia de datos	12
SIA-12. Excepciones propias y try-catch	13
SIA-13. Uso de GitHub	14

SIA-1 Análisis de datos y funcionalidades

Requisito solicitado: Realizar un análisis de los datos a utilizar y de las principales funcionalidades que dan sentido al proyecto.

Análisis realizado

El Sistema de gestión de asistencia administra cursos, alumnos y asistencia. Un alumno puede estar matriculado en un curso. Mientras que la asistencia debe indicar si el alumno estuvo presente, ausente, tiene una ausencia justificada o si ha sido retirado antes de tiempo. Estos requisitos permiten al colegio llevar un registro de la asistencia, consultarla y gestionarla, así como registrar alumnos y cursos. Además, se podrá consultar asistencia por alumnos y cursos.

Evidencia

Archivo: [README.md](#), sección “La idea”: explica el objetivo general del sistema, la cual fue pensada al momento de crear el repositorio.

SIA-2 Diseño conceptual de clases

Requisito solicitado: Diseñar el modelo conceptual de clases del dominio mediante UML y codificarlo en Java.

Clases del dominio

1. **Persona:** contiene datos comunes de las personas en el sistema (RUT, nombre y apellido)
2. **Alumno:** contiene una referencia al curso al que pertenece
3. **Curso:** contiene los datos de código, nombre, profesor jefe y lista de alumnos del curso.
4. **Asistencia:** es un registro de asistencia de un alumno en una fecha determinada, contiene presencia, falta justificada, estado de salida anticipada y comentario.

Relaciones

Se hace uso de herencia para compartir los atributos de Persona con Alumno, entre Curso y Alumno existe una asociación en ambos sentidos dado que el curso mantiene una lista de alumnos y Alumno mantiene una referencia a su curso.

Entre la clase Alumno y Asistencia existe una asociación donde cada registro de asistencia corresponde a un alumno y cada alumno puede tener múltiples registros, Asistencia guarda la referencia alumno.

Evidencia UML

[Diagrama UML Clases Dominio.pdf](#)

Evidencia principal:

[src/main/java/Persona.java](#)

[src/main/java/Curso.java](#)

[src/main/java/Asistencia.java](#)

[src/main/java/Alumno.java](#)

Evidencia principal: [src/main/java](#)

SIA-3 Buenas prácticas

Requisito solicitado: Mantener atributos privados con getters y setters, código legible y modularizado, documentación interna útil y datos iniciales suficientes.

Encapsulamiento

Utilizamos encapsulamiento declarando los atributos de las clases como private y accediendo a ellos usando métodos públicos.

Organización del código

El proyecto lo dividimos según la responsabilidad de cada clase. Las clases del dominio representan datos principales del sistema, mientras que otras clases como por ejemplo GestionAlumnos o GestionCursos tienen las operaciones para realizar sobre las clases principales y sus datos. Las clases de Menú se encargan de la navegación por consola y las clases Ventana corresponden a nuestra interfaz gráfica.

Datos iniciales

Nuestros datos iniciales son creados en DatosIniciales.java, estos crean objetos de tipo Curso, Alumno y Asistencia. Nos permiten iniciar el programa con información de prueba para ejecutar funcionalidades sin tener que ingresar todos los datos manualmente cada vez que iniciemos el proyecto.

Evidencia principal:

[src/main/java/Asistencia.java](#), líneas aproximadas 4-10 y 32-94.

[src/main/java/Alumno.java](#), líneas aproximadas 3-19.

[src/main/java/GestionAlumnos.java](#), líneas aproximadas 13-36.

[src/main/java/DatosIniciales.java](#), líneas aproximadas 7-29.

Evidencia principal: [src/main/java](#)

SIA-4 Colecciones anidadas

Requisito solicitado: Diseñar y codificar dos colecciones de objetos, con la segunda anidada; al menos una debe ser un mapa del Java Collections Framework.

Colecciones utilizadas y Anidamiento

Nuestro proyecto usa un HashMap para guardar los cursos y un ArrayList para guardar los alumnos de cada curso, en GestionCursos.java por ejemplo podemos observarlo como:

```
private final HashMap<String, Curso> cursos = new HashMap<>();
```

Aquí la clave del mapa es el código del curso y el valor corresponde a un objeto de tipo Curso.

Después podemos observar que en nuestro archivo Curso.java cada curso tiene una colección de alumnos la cual se declara como

```
private final ArrayList<Alumno> alumnos;
```

Por lo que el ArrayList de alumnos está anidado dentro de los objetos Curso los cuales están dentro del HashMap de Gestión Cursos.

Evidencia

[src/main/java/GestionCursos.java](#), aproximadamente líneas 8 y 18.

[src/main/java/Curso.java](#), aproximadamente líneas 9 y 15.

[src/main/java/GestionRegistroAsistencia.java](#), aproximadamente línea 10.

Evidencia principal: [GestionCursos.java](#), [Curso.java](#) y [GestionRegistroAsistencia.java](#).

SIA-5 Sobrecarga de métodos

Requisito solicitado: Diseñar y codificar dos clases que utilicen sobrecarga de métodos, sin considerar sobrecarga de constructores. Ambas clases deben ser utilizadas.

Clases y métodos

1. **GestionAlumnos:** contiene dos versiones del método **buscarAlumno**, una que recibe un RUT y otra que recibe un nombre y apellido. Ambos métodos buscan un Alumno en la lista general de alumnos y lo retornar o null si no existe.
2. **Alumno:** contiene dos versiones del método **mostrarResumen**, una sin parámetros, que imprime los datos del alumno en varias líneas y otra que recibe un booleano, el cual permite elegir un formato corto de impresión de datos para fines estéticos y de legibilidad.

Uso

buscarAlumno: es llamado desde el método **obtenerAlumno** que utiliza el menú cuando el usuario busca un alumno por RUT, también se utiliza como método de verificación en el agregado de alumnos, la versión que recibe nombre y apellido es llamada por el método **mostrarMenu** en la clase **MenuAlumnos** y desde **VentanaAlumnos**.

mostrarResumen: es llamado desde **MenuAlumnos** para mostrar el resultado de una búsqueda, la versión de formato corto se utiliza desde el método **mostrarAlumnos** en la clase **GestionAlumnos**.

Evidencia

- [src/main/java/GestionAlumnos.java](#), **buscarAlumno()** aproximadamente linea 25-38
- [src/main/java/Alumno.java](#), **mostrarResumen()** aproximadamente linea 22-42

Evidencia principal: [Alumno.java](#), [GestionAlumnos.java](#), [MenuAlumnos.java](#)

SIA-6 Sobreescritura de métodos

Requisito solicitado: Diseñar y codificar dos clases que utilicen sobreescritura de métodos. Ambas clases deben ser utilizadas.

Clases y Sobreescritura

En la clase persona podemos ver que definimos el método mostrarResumen() y luego en las clases Alumno y Profesor se sobrescriben utilizando @Override, cada una de estas clases entrega un comportamiento diferente para el mismo método aprovechándose de la herencia de Persona.

Evidencia de uso

- [src/main/java/Persona.java](#), aproximadamente líneas 38-43.
- [src/main/java/Alumno.java](#), aproximadamente líneas 21-40.
- [src/main/java/Profesor.java](#), aproximadamente líneas 20-39.

Evidencia principal: [Persona.java](#), [Alumno.java](#) y [Profesor.java](#)

SIA-7 Menú de inserción y listado

Requisito solicitado: Crear un menú del sistema con inserción manual y listado para ambas colecciones, en opciones separadas.

Menú de consola

Nuestro menú cuenta con una sección de administración la cual permite acceder a la gestión de alumnos y cursos.

En MenuAlumnos.java se presentan las opciones de:

- Agregar un alumno
- Mostrar un alumno
- Mostrar todos los alumnos

En MenuCursos.java se presentan las opciones de:

- Agregar un curso
- Mostrar un curso

Ventanas

Las mismas funcionalidades mencionadas anteriormente también están en nuestras ventanas realizadas en netbeans en forma de Jforms con Swing.

Evidencia

- [src/main/java/MenuAlumnos.java](#), aproximadamente líneas 23-29 y 40-94.
- [src/main/java/MenuCursos.java](#), aproximadamente líneas 20-56.
- [src/main/java/VentanaAlumnos.java](#).
- [src/main/java/VentanaCursos.java](#)

Evidencia principal: [MenuAlumnos.java](#), [MenuCursos.java](#), [VentanaAlumnos.java](#) y [VentanaCursos.java](#).

SIA-8 Menú de edición eliminación y búsqueda

Requisito solicitado: Crear opciones separadas para editar, eliminar y buscar elementos de ambas colecciones.

Operaciones implementadas

1. **Buscar alumno:** busca un alumno mediante su RUT o nombre y apellido, con el método `obtenerAlumno`, retorna el objeto encontrado.
2. **Editar alumno:** solicita el RUT, busca el alumno y modifica su nombre y/o apellido mediante uso de setters heredados de `persona`, RUT no es modificable.
3. **Eliminar alumno:** busca un alumno mediante su RUT, si pertenece a un curso, lo elimina de la lista de alumnos del curso y después lo elimina de la lista de alumnos global.
4. **Buscar curso:** utiliza el código como key para el `HashMap` de cursos, si lo encuentra, muestra sus datos.
5. **Editar curso:** busca el curso por código, modifica su nombre y profesor jefe, el código no es modificable.
6. **Eliminar curso:** busca el curso por código, antes de eliminarlo, asigna null al curso de cada alumno de su lista de alumnos, luego lo elimina del `HashMap` de cursos.

Validaciones

Los métodos `obtenerAlumno` y `obtenerCurso` cuentan con excepciones propias, `AlumnoNoEncontradoException` y `CursoNoEncontradoException`, respectivamente, los menús capturan estas excepciones y muestran un mensaje correspondiente sin cerrar el programa.

En las ventanas se aplica validación de input con `trim`, los métodos de edición y eliminación retorna false en caso de no encontrar el elemento a eliminar, la búsquedas retornan null en caso de no encontrar coincidencias.

Los inputs inválidos para las opciones de los menús de consola se controlan mediante `NumberFormatException`.

Evidencia

- `src/main/java/VentanaAlumnos.java` aproximadamente linea 276-317
- `src/main/java/VentanaCursos.java` aproximadamente linea 176-220
- `src/main/java/MenuCursos.java` aproximadamente linea 75-80
- `src/main/java/MenuAlumnos.java` aproximadamente linea 74-87
- `src/main/java/GestionCursos.java` aproximadamente linea 23-100
- `src/main/java/GestionAlumnos.java` aproximadamente linea 25-131

Evidencia principal: [GestionAlumnos.java](#), [GestionCursos.java](#)

SIA-9 Funcionalidad de utilidad para el negocio

Requisito solicitado: Implementar al menos una funcionalidad útil para el negocio, distinta de inserción, edición, eliminación o reportes, considerando un subconjunto filtrado por criterio.

Funcionalidad

Nuestro programa permite obtener alumnos que estuvieron ausentes en una fecha específica.

El usuario puede ingresar una fecha y nuestro programa recorre todos los registros. Luego aplica un filtro en el que conserva solo los que cumplen con que la fecha coincida con la ingresada y que el alumno no esté marcado como presente.

Luego de este filtrado, se muestra el RUT y el nombre de cada alumno ausente en esa fecha. Si ningún registro cumple muestra un mensaje diciendo que no hay alumnos ausentes en dicha fecha.

Utilidad

Esta funcionalidad es útil para que un colegio pueda identificar de manera rápida que alumnos faltaron durante un día específico y realizar el seguimiento correspondiente.

Evidencia

Tenemos el filtrado en `GestionRegistroAsistencia.java`, está en el método `obtenerAusentesPorFecha()`, líneas 239 y 264 aproximadamente.

La opción está tanto en ventana como en menú de consola.

[GestionRegistroAsistencia.java](#), método `obtenerAusentesPorFecha()`.

[MenuAsistencia.java](#), opción de alumnos ausentes por fecha.

[VentanaAsistencia.java](#), consulta mediante interfaz gráfica.

[SIA-9: Mostrar alumnos ausentes por fecha #40](#).

SIA-10 Consola y ventanas

Requisito solicitado: Implementar todas las funcionalidades mediante consola y ventanas, solicitando al inicio el modo de uso.

Selección inicial

El programa comienza en **GestionAsistencia.main()**, donde se crea una instancia de cada clase gestora, instancias que pasa a la clase **Menu**, este muestra tres opciones (usar consola, usar ventana, salir), si el usuario selecciona la consola se llama al método **mostrarMenu** y comienza la navegación por consola, si el usuario selecciona la ventana, se crea una instancia de **VentanaPrincipal**, se le entregan las clases gestoras y se muestra al usuario mediante **setVisible(true)**, finalmente el usuario puede terminar el menú y con ello, el programa.

Paridad funcional

La funcionalidad esperada está disponible en ambas interfaces, esto es comprobable al ver que las clases de menú y las clases de ventana correspondientes (**MenuAlumnos**, **VentanaAlumnos**, **MenuCursos**, **VentanaCursos**, **MenuAsistencia**, **VentanaAsistencia**, **MenuAvanzado**, **VentanaAvanzado**) utilizan y tienen acceso a los mismos métodos de las clases de gestión (**GestionCursos**, **GestionAlumnos**, **GestionAsistencia**, **GestionRegistroAsistencia**).

Evidencia

- [src/main/java/GestionAsistencia.java](#) aproximadamente línea 7-32
- [src/main/java/Menu.java](#) aproximadamente línea 27-61
- [src/main/java/MenuPrincipal.java](#) aproximadamente línea 10-19
- [src/main/java/VentanaPrincipal.java](#) aproximadamente línea 15-34
- [src/main/java/VentanaAdministracion.java](#) aproximadamente línea 18-48

Evidencia principal: [GestionAsistencia.java](#), [Menu.java](#), [MenuPrincipal.java](#), [VentanaPrincipal.java](#), [VentanaAdministracion.java](#)

SIA-11 Persistencia de datos

Requisito solicitado: Persistir datos mediante archivo de texto, CSV, Excel o DBMS local, cargando al inicio y guardando al salir mediante un sistema batch.

Formato y archivos

El sistema utiliza archivos de texto en formato **CSV**, en estándar UTF-8, estos archivos se encuentran en la carpeta **datos**, ubicada en la raíz del proyecto.

1. **datos/cursos.csv**: almacena el código, nombre y profesor jefe de cada curso.
2. **datos/alumnos.csv**: almacena el RUT, nombre, apellido y código del curso asignado a cada alumno.
3. **datos/asistencia.csv**: almacena el RUT del alumno, fecha, presencia, retiro anticipado, falta justificada y los motivos relacionados.

Se hace uso de **Path**, **Files**, **BufferedReader** y **BufferedWriter**, junto con un método **formatear** y **limpiar** para control de formato.

Carga inicial

En el inicio, **GestionAsistencia.main()** crea los gestores y construye un objeto **CSV**, donde luego ejecuta el método **cargarTodo**, el cual crea la carpeta **datos** si es que no existe aún y realiza la carga de datos en las carpetas.

El método **cargarTodo**, llama a otros tres métodos, **cargarAlumnos**, **cargarCursos** y **cargarAsistencias**, los cuales recuperan los datos de los archivos csv y los pasan como parámetros a los métodos de agregado (**agregarCurso**, **agregarAlumno**) los cuales se encargan de crear las instancias, con los datos correspondientes.

Guardado final

En **GestionAsistencia.main()** se registra un **shutdownHook**, este bloque se ejecuta cuando el programa finaliza de forma normal y llama al método **guardarTodo**, el cual crea la carpeta **datos** si no existe y llama a tres otros tres métodos, **guardarAlumnos**, **guardarCursos** y **guardarAsistencias**, los cuales abren su archivo respectivo con **BufferedWriter** y se reescriben los archivos csv con los datos contenidos en las colecciones de la sesión actual, lo cual permite incluir los cambios que se hayan realizado a las colecciones.

Evidencia

- [src/main/java/CSV.java](#) aproximadamente línea 6-202
- [src/main/java/GestionAsistencia.java](#) aproximadamente línea 8-29
- [datos/cursos.csv](#)
- [datos/alumnos.csv](#)
- [datos/asistencias.csv](#)

Evidencia principal: [CSV.java](#), [GestionAsistencia.java](#)

SIA-12 Excepciones propias y try catch

Requisito solicitado: Diseñar y codificar dos excepciones que se utilicen en el programa. Manejar las excepciones mediante try catch.

Excepciones

1. **AlumnoNoEncontradoException:** se utiliza cuando una búsqueda de alumno por RUT no encuentra al alumno determinado, es subclase de **Exception**.
2. **CursoNoEncontradoException:** se utiliza cuando una búsqueda por código no encuentra el curso determinado, es subclase de **Exception**.

Manejo

En consola, los menús son los responsables de capturar las excepciones, estos colocan la lectura y ejecución de las opciones dentro de bloques **try-catch**, el mensaje se imprime por consola.

En ventana, las ventanas **Swing** son las responsables de capturar las excepciones dentro de los métodos asociados a los botones, el mensaje es mostrado en un cuadro de diálogo.

Recuperación

En consola, los bloques **try-catch** de los menus estan dentro de ciclos **do-while**, por lo que cuando ocurre un error recuperable, el bloque **catch** muestra el mensaje por consola y la ejecución llega al final de ciclo, el menú vuelve a comenzar y se solicita una opción nuevamente, evitando cerrar el programa.

En ventana, la excepción solo termina la acción del botón que se está ejecutando, la ventana permanece abierta dándole la oportunidad al usuario de corregir el dato o cambiar de operación.

Evidencia

- [src/main/java/AlumnoNoEncontradoException.java](#) aproximadamente linea 2-7
- [src/main/java/CursoNoEncontradoException.java](#) aproximadamente linea 2-7
- Los menús correspondientes, están envueltos en bloque try-catch.

Evidencia principal: [CursoNoEncontradoException.java](#), [AlumnoNoEncontradoException.java](#)

SIA-13 Uso de GitHub

Requisito solicitado: Utilizar GitHub realizando al menos seis commits.

Historial

La rama **main** contiene 100 commits, desde documentación, correcciones, implementación y merges, estos commits están registrados desde el 9 de agosto de 2026 y llegan hasta el 9 de septiembre de 2026.

Organización

El trabajo fue dividido en **issues** semanales acorde al avance SIA recomendado en la planificación del curso.

El repositorio mantiene cuatro ramas, la rama **main** la cual contiene la versión integrada y compartida del proyecto, la rama **nicolasj** la cual es la rama personal utilizada por Nicolas Jaramillo, la rama **lucasahu** la cual es la rama personal utilizada por Lucas Ahumada y la rama **alessandror** la cual es la rama personal de Alessandro Reginato.

Cada integrante desarrolló sus **issues** en su propia rama, una vez completado, se integraba a la rama **main** por medio de **pull request** y **merge**, este flujo permite trabajar de forma paralela, evitando introducir cambios disruptivos para el código de la rama **main**.

Evidencia

Evidencia principal: [Repositorio de GitHub](#), [Issues](#), [Ramas](#), [Historial de commits](#).

